

Checklist de Revisão — Equipe SIGA

Usado obrigatoriamente pelo **Modo 3** e disponível como referência para os demais após a atividade. É um checklist **genérico da equipe** — ele não aponta os defeitos, ele orienta o olhar.

Seção 1 — Conformidade com o requisito (*Revisor A*)

- [] **1.1** Cada regra de negócio do requisito tem um trecho de código correspondente? Aponte qual.
- [] **1.2** Existe alguma regra do requisito **sem nenhuma** implementação no código?
- [] **1.3** Os valores constantes do código (limites, prazos, quantidades) batem com os números do documento — **inclusive com a versão mais recente dele**?
- [] **1.4** Alguma regra foi implementada apenas **parcialmente** (um caso tratado, outro esquecido)?
- [] **1.5** Cada critério de aceitação passaria neste código? Percorra um a um.
- [] **1.6** O requisito contém pontos **ambíguos ou não verificáveis**? (isso também é defeito — do requisito)

Seção 2 — Lógica e casos de borda (*Revisor A*)

- [] **2.1** Comparações de fronteira: `<` vs `<=`, `>` vs `>=` — o limite está incluído ou excluído corretamente?
- [] **2.2** O que acontece com entrada vazia, nula, zero ou negativa?
- [] **2.3** Existe verificação de uma condição seguida de ação sobre ela (*check-then-act*)? O estado pode mudar entre as duas?
- [] **2.4** Uma condição está sendo testada por um **proxy** em vez do fato real (ex.: "existe registro" quando o que importa é "registro com resultado X")?
- [] **2.5** Todos os caminhos retornam algo coerente com o contrato do método?

Seção 3 — Segurança e privacidade (*Revisor B*)

- [] **3.1** Todas as consultas ao banco são **parametrizadas**? Há concatenação de entrada em SQL?
- [] **3.2** Logs, mensagens de erro e exceções vazam **dado pessoal, credencial ou token**?
- [] **3.3** Há validação da entrada antes de usá-la?
- [] **3.4** Falhas são tratadas de forma que **não escondam** o erro?
- [] **3.5** O código respeita as restrições não funcionais declaradas (LGPD, padrão de codificação)?

Seção 4 — Testes (*Revisor B*)

- [] **4.1** Cada regra de negócio tem pelo menos um teste?
- [] **4.2** Os testes verificam **comportamento** ou apenas que "não explodiu"? (`assert x is not None` prova o quê?)
- [] **4.3** Existem testes para os caminhos de **falha e recusa**, não só o caminho feliz?
- [] **4.4** Algum teste está **congelando um comportamento errado** — isto é, foi escrito a partir do código em vez do requisito?
- [] **4.5** Os dublês de teste (mocks/fakes) escondem o comportamento real que importa?

Seção 5 — Manutenibilidade e legibilidade (*Revisor C*)

- [] **5.1** Há números mágicos? Alguma constante aparece **duplicada com valores diferentes**?
- [] **5.2** Nomes de variáveis, métodos e parâmetros dizem o que a coisa é?
- [] **5.3** O contrato de retorno é consistente (o método sempre devolve o mesmo tipo/forma)?
- [] **5.4** Há código morto, campo declarado e nunca usado, ou comentário que mente?

- [] **5.5** Um desenvolvedor novo entenderia este método sem perguntar ao autor?
-

Regras de uso do checklist

1. **Marque o item mesmo quando estiver tudo certo.** Item não percorrido é área não revisada.
2. **Um item pode gerar mais de um defeito.** Não pare no primeiro.
3. **Checklist não substitui pensar.** Ele garante o piso, não o teto.